

Docker Cheatsheet

面向机器学习项目的 Docker 运维速查表

运行新容器

从镜像启动新容器 (格式: docker run [OPTIONS] IMAGE [COMMAND])

```
docker run nginx
```

指定容器名称

```
docker run --name web nginx
```

映射端口 宿主机:容器 (-P 映射所有暴露端口)

```
docker run -p 8080:80 nginx
```

指定容器主机名

```
docker run --hostname svr nginx
```

添加 DNS 记录 (主机名:IP)

```
docker run --add-host db:192.168.1.1 nginx
```

挂载本地目录到容器

```
docker run -v /宿主机路径:/容器路径 nginx
```

后台运行且保留交互 (-i 交互 -t 伪终端 -d 后台)

```
docker run -itd nginx
```

覆盖镜像默认入口命令

```
docker run --entrypoint bash nginx
```

管理容器

列出正在运行的容器 (-a 含已停止)

```
docker ps
```

列出所有容器

```
docker ps -a
```

删除已停止的容器 (-f 强制删除运行中)

```
docker rm web
```

删除所有已停止的容器

```
docker container prune
```

停止容器

```
docker stop web
```

启动已停止的容器

```
docker start web
```

从容器复制到宿主机 (容器:源 → 目标)

```
docker cp web:/index.html .
```

从宿主机复制到容器

```
docker cp index.html web:/index.html
```

在运行中容器内启动 shell (进入容器)

```
docker exec -it web bash
```

重命名容器

```
docker rename 096 web
```

从容器创建新镜像

```
docker commit web
```

数据与卷（持久化重点）

查看所有 Docker 卷
(推荐用卷，数据（数据集、模型、长期日志）——优先放卷：稳定、可移植、易于备份和多机统一。)

```
docker volume ls
```

创建卷，用来存放训练数据/模型等持久化内容

```
docker volume create ml-data
```

将卷挂载到容器内 `/data`，容器删掉数据仍在

```
docker run -v ml-data:/data my-ml-image
```

直接把当前目录映射进容器：适合保存训练中间结果/日志

```
docker run -v $PWD/checkpoints:/checkpoints my-ml-image
```

查看卷实际在宿主机上的路径（排错/备份时有用）

```
docker inspect ml-data
```

用一次性容器检查卷内数据情况（不要直接手改宿主机路径）

```
docker run --rm -v ml-data:/data alpine ls -lh /data
```

容器网络配置

查看现有网络 (bridge / host / none / 自定义网络)

```
docker network ls
```

查看指定网络详情 (子网 网关 已连接容器)

```
docker network inspect bridge
```

创建自定义 bridge 网络 (容器可互相通过名称访问)

```
docker network create mynet
```

在指定网络中启动容器 (同网容器可通过名称互访)

```
docker run --network mynet --name web nginx
```

将已存在容器连接到网络 (多网卡场景)

```
docker network connect mynet db
```

将容器从某个网络移除

```
docker network disconnect mynet db
```

使用宿主机网络 (无端口映射 直接占用宿主机端口)

```
docker run --network host nginx
```

完全隔离网络 (无网络访问)

```
docker run --network none nginx
```

GPU 与资源限制

检查容器是否能看到 GPU（前提：宿主机已安装驱动 + nvidia-docker）

```
docker run --gpus all nvidia/cuda:12.2.0-base nvidia-smi
```

只暴露指定编号 GPU，防止容器占掉全部卡

```
docker run --gpus '"device=0,1"' my-ml-image
```

限制容器内最大内存和 CPU，防止 ML 任务吃满整机

```
docker run -m 16g --memory-swap 16g \
--cpus 8 my-ml-image
```

在线调整某个已存在容器的资源限制（视 Docker 版本而定）

```
docker update --cpus 4 --memory 8g my-ml-container
```

增大 /dev/shm，解决 PyTorch / Dataloader 因共享内存不足报错

```
docker run --shm-size=16g my-ml-image
```

管理镜像

拉取镜像 (可带 :TAG)

```
docker pull nginx
```

推送镜像到仓库

```
docker push myimage:1.0
```

删除镜像

```
docker rmi nginx
```

列出本地镜像

```
docker images
```

删除虚悬镜像 (-a 删除所有未使用)

```
docker image prune
```

从 Dockerfile 构建镜像并打标签

```
docker build -t myimage .
```

从当前目录 Dockerfile 构建 (无标签)

```
docker build .
```

给镜像打标签

```
docker tag ubuntu ubuntu:18.04
```

将镜像导出为 .tar 文件

```
docker save nginx > nginx.tar
```

从 .tar 文件加载镜像

```
docker load -i nginx.tar
```

信息与统计

查看容器日志 (-f 实时跟踪)

```
docker logs web
```

查看运行中容器的 CPU/内存等统计

```
docker stats
```

查看容器内进程列表

```
docker top web
```

查看 Docker 版本

```
docker version
```

查看容器/镜像详细信息 (JSON)

```
docker inspect web
```

查看容器内相对镜像的文件变更

```
docker diff web
```

查看容器端口映射

```
docker port web
```

常见排错套路

容器刚启动就退出 / 训练中断：第一步先看完整日志

```
docker logs -f 容器名
```

怀疑配置/挂载/网络有问题：看 inspect 中是否有明显错误字段

```
docker inspect 容器名 | grep -i error
```

问题定位不到：直接进入容器里手动跑命令重现

```
docker exec -it 容器名 bash
```

训练变慢/卡死：查看是否 CPU/GPU/内存打满，容器是否在 OOM 前挣扎

```
docker stats
```

训练进程莫名其妙被杀：看宿主机内核日志是否有 OOM killer 记录

```
dmesg | tail -n 50
```

磁盘快满：查看镜像/容器/卷各占多少空间

```
docker system df
```

清理未使用的容器/镜像/网络（危险：会删掉在用但未来可能要用的镜像）

```
docker system prune -a
```

镜像仓库与版本管理

登录私有镜像仓库（CI/CD 或多机训练前必做）

```
docker login registry.example.com
```

按日期/版本号给 ML 镜像打可追踪的 tag，方便回滚

```
docker tag my-ml-image:latest \
registry.example.com/ml/my-ml-image:2026-03-12
```

推送镜像到私有仓库，在多台训练机/生产环境复用

```
docker push registry.example.com/ml/my-ml-image:2026-03-12
```

在其他机器上拉取同一版本镜像，确保训练环境一致

```
docker pull registry.example.com/ml/my-ml-image:2026-03-12
```